

Smart City Transportation Network Optimization

Technical Report

Project: Cairo Smart City Transportation Network Optimization

Course: CSE112 - Design and Analysis of Algorithms

Platform: Streamlit Web Application with Docker Support

1. Executive Summary

The Cairo Smart City Transportation Network Optimization project is an integrated software system designed to model, analyze, and improve transportation decisions across a simplified Greater Cairo road network. The project combines classical graph algorithms, dynamic programming, greedy optimization, machine learning, and interactive visualization into one Streamlit web application.

The system allows users to explore route planning, emergency response routing, infrastructure expansion, public transit allocation, road maintenance planning, and network health monitoring from a single interface. The project uses a graph-based representation of Cairo. Nodes represent districts, neighborhoods, and facilities such as hospitals, airports, universities, railway stations, and business areas. Edges represent roads between these nodes and store distance, capacity, condition, and construction cost.

During the modification phase, the main issue we faced was that the Maintenance DP map did not visually behave like the rest of the project maps. It appeared as a separate graph-style visualization instead of a real interactive Cairo map. It also needed consistent zoom behavior with the other maps. We fixed this by integrating the maintenance result into the shared Plotly Mapbox renderer and enabling scroll zoom for all real maps.

2. Problem Statement And Objectives

Urban transportation systems involve many competing decisions. A city may need to find the fastest route between two points, choose which roads to build, decide how to allocate public buses, repair damaged roads within a limited budget, and improve traffic signal behavior during congestion. These problems cannot all be solved using the same algorithm because each has a different structure and objective.

- Represent Cairo as a weighted transportation graph using nodes and edges.
- Apply suitable algorithms to real transportation decisions.
- Provide an interactive Streamlit interface for non-technical users.
- Visualize decisions on a real Cairo basemap rather than abstract graphs only.
- Support reproducible execution through Docker.

The project focuses on road expansion, shortest-path routing, emergency routing, public

transit allocation, maintenance prioritization, signal optimization, and traffic prediction. Each feature demonstrates a specific algorithmic technique and connects it to a practical smart-city use case.

3. System Architecture And Data Model

The system is organized around a shared graph model. The main data files are nodes.csv, edges.csv, Traffic_Flow_Patterns.csv, and traffic_model.pkl. The graph engine is implemented in data_loader1.py and defines Node, Edge, and CairoGraph classes.

Each node includes an ID, name, type, population, longitude, and latitude. Each edge includes source node, destination node, distance, capacity, condition, and construction cost. Existing roads have zero construction cost, while potential future roads have a positive construction cost.

The Streamlit application loads the graph once using a cached function. This improves performance because the graph and machine learning predictor do not need to be rebuilt on every page interaction. The same graph object is then passed into the Trip Planner, Emergency Response, Infrastructure, and System Dashboard pages.

- app.py: Streamlit user interface and shared Plotly map renderer.
- data_loader1.py: graph engine for nodes, edges, and adjacency lists.
- dijkstra.py and astar.py: routing modules.
- kruskal_mst.py: infrastructure expansion planning.
- dp_maintenance.py: road repair optimization using DP Knapsack.
- public_transit_scheduler.py: bus allocation using Dynamic Programming.
- greedy.py: traffic signal optimization.

4. Algorithms Used

4.1 Dijkstra's Algorithm

Dijkstra's algorithm is used to find the shortest or fastest path between two nodes. It is suitable because all road weights are non-negative. In this project, edge weights may represent distance or predicted travel time depending on whether the traffic predictor is active.

4.2 A* Search

A* Search is used for emergency routing. It improves search efficiency by using a heuristic based on geographic distance to the destination. This helps emergency vehicles find routes while exploring fewer nodes than Dijkstra in many cases.

4.3 Kruskal's Minimum Spanning Tree

Kruskal's algorithm is used for road expansion. Existing roads are treated as already connected, and potential roads are sorted by construction cost. A Union-Find structure prevents cycles. This helps select cost-effective new roads that improve city connectivity.

4.4 Dynamic Programming For Bus Allocation

The public transit scheduler uses Dynamic Programming to allocate a limited bus fleet across routes. This is better than a simple greedy method because it evaluates many allocation combinations and avoids assigning too many buses to a single route.

4.5 DP Knapsack For Road Maintenance

The maintenance module converts each existing road into a repair candidate with repair cost and repair value. The problem becomes a 0/1 Knapsack problem: choose a subset of roads to repair without exceeding the budget while maximizing total score gain.

4.6 Greedy Signal Optimization

The greedy signal optimizer models local traffic-light decisions by choosing the action with the best immediate benefit. It is fast and useful for real-time demonstrations, although it may not always produce a city-wide optimal result.

5. Main Problem Faced

The main problem appeared in the Infrastructure page, specifically in the Maintenance DP tab. The maintenance optimizer itself worked correctly. For example, with a budget of 2,000 cost-units, the optimizer selected 12 roads, used 1,973 cost-units, and produced a score gain of +156. However, the visualization did not match the rest of the project.

The rest of the application used real interactive Cairo maps through Plotly Mapbox. Trip planning, emergency response, and infrastructure expansion were displayed geographically using longitude and latitude. In contrast, the maintenance DP visualization appeared more like an abstract graph or separate heatmap.

This caused an inconsistent user experience. Users could see real map behavior on other pages but not in the maintenance section. It also made the maintenance result harder to understand because road repair decisions are spatial and should be shown on the actual city map.

A second issue was zoom behavior. After the maintenance map was converted into a real map, the user also needed to zoom in and out of it like the other maps. Then the requirement expanded: zoom should be enabled consistently across all maps, not only the maintenance map.

6. How We Fixed It

6.1 Reusing The Shared Map Renderer

The fix was implemented in app.py. Instead of displaying Maintenance DP with a separate visualization, the shared `build_map()` function was extended with an optional `maintenance_result` parameter. When this parameter is provided, the map enters maintenance mode while still using the same Cairo basemap, node rendering, layout, and Plotly behavior as the rest of the app.

Roads are colored by condition: poor roads are shown in red or pink, fair roads in yellow, good roads in green, and DP-selected roads in cyan. This made the maintenance result visually consistent with Trip Planner, Emergency Response, and MST Infrastructure maps.

6.2 Highlighting Selected Repair Roads

Selected repair roads are identified from the DP result by storing each selected road as a sorted pair of endpoint IDs. If an edge appears in this selected set, it is drawn with a thicker cyan line and a glow layer. This makes the algorithm's decision easy to see on the dark basemap.

The hover text was also improved. Users can hover over a road to see condition, distance, repair cost, score gain, and whether it was selected for repair.

6.3 Enabling Zoom On All Maps

To make zoom behavior consistent, one shared Plotly map configuration was added: `PLOTLY_MAP_CONFIG = {"displayModeBar": False, "scrollZoom": True}`. This configuration was applied to the Trip Planner map, Emergency Response map, Road Expansion MST map, and Maintenance DP map. Non-map bar charts were left unchanged.

This solved the usability issue while keeping the clean hidden-toolbar interface used by the project.

7. Dockerization And Deployment Readiness

The project is Docker-ready. The repository includes a Dockerfile, requirements.txt, and .dockerignore. The Dockerfile uses python:3.11-slim, installs required system packages, installs Python dependencies, copies the project files, configures Streamlit, exposes port 8501, and runs the application.

The important startup command is: `streamlit run app.py --server.port=${PORT:-8501} --server.address=0.0.0.0`. This allows the app to run locally on port 8501 while also supporting cloud platforms that provide a PORT environment variable.

To build and run the Docker image after the modifications, use: `docker build -t smart-city-transportation`. Then run: `docker run --rm -p 8501:8501 smart-city-transportation`. The app can then be opened at `http://localhost:8501`.

The Dockerized app includes the updated app.py with the real maintenance map and zoom enabled on all maps.

8. Testing And Verification

The modified code was verified using Python compilation checks: `python -m py_compile app.py dp_maintenance.py`. The Streamlit app was also run locally and checked through the browser.

- The app launched successfully on localhost.
- The Infrastructure page opened correctly.
- The Maintenance DP optimizer ran successfully.
- Maintenance KPI cards updated correctly.
- The maintenance result appeared on a real Cairo basemap.
- Selected repair roads were highlighted in cyan.
- Zoom was enabled on all map views.
- Non-map charts were not changed unnecessarily.

9. Conclusion

The Cairo Smart City Transportation Network Optimization project successfully integrates multiple algorithmic techniques into one practical transportation dashboard. Dijkstra and A* solve routing problems, Kruskal's algorithm supports infrastructure planning, Dynamic Programming solves bus allocation and maintenance planning, a greedy algorithm models signal control, and machine learning supports traffic prediction.

The most important improvement made during this modification was fixing the Maintenance DP visualization. The algorithm was already correct, but the result needed to be shown in a way that matched the rest of the system. By integrating maintenance into the shared Plotly map renderer and enabling zoom across all maps, the project became more consistent, professional, and easier to use.

The final version is locally saved, Docker-ready, and improved with a real interactive maintenance map and consistent map zoom behavior across the application.